实践手册 · RTX 4070 12GB 版

模型蒸馏、工作流与 MCP 实战教程

从“训练出一个更小的模型”，到“编排一个可靠的 Agent”，再到“用标准协议把工具接进来”。目标不是堆概念，而是四周内做出能演示、能部署、能报价的作品。

01 蒸馏

白盒 KL + 黑盒合成数据

02 工作流

LangGraph + n8n + Dify

03 MCP

FastMCP + 客户端接入

简体中文 · 原理 + 可运行代码 + 实战步骤

2026-10-08

这条路线把模型能力、流程编排与标准接口串成产品

蒸馏解决“模型太大、太慢、太贵”；工作流解决“模型行为不可控”；MCP（Model Context Protocol，模型上下文协议）解决“每接一个工具都要写一套胶水”。三者组合后，才是客户愿意验收的 AI 应用。

教师模型 能力来源 → **学生模型** 低成本推理 → **LangGraph** 状态与控制 → **MCP Server** 工具与数据

硬件边界

- 单张 RTX 4070 12GB。
- 白盒蒸馏从 1.7B → 0.6B 起步。
- 序列长度 512，batch 1，梯度累积。
- 7B 教师在线蒸馏不作为首个实验。

最终作品

- 蒸馏实验报告与基线对比。
- 客服订单 Agent 的 LangGraph 图。
- 订单查询 MCP Server。
- 一键演示脚本、架构图、验收清单。

建议节奏：你每周约三天可投入，按“1 天理解与搭骨架 + 1 天实现 + 1 天评测与整理作品集”推进。先跑通，再扩数据与模型规模。

目录

01	环境与验收基线	3
02	白盒 / 黑盒模型蒸馏	4-9
03	LangGraph、n8n 与 Dify 工作流	10-15
04	MCP 原理、Server 与接入	16-21
05	四周计划、交付与参考资料	22-24

先准备可复现且不会泄露密钥的实验环境

最低准备

- Ubuntu 22.04/24.04 或 WSL2；NVIDIA 驱动能执行 `nvidia-smi`。
- Python 3.11、Git、uv；剩余磁盘建议 40GB 以上。
- 训练机：RTX 4070 12GB。VPS 只负责 API / MCP / 工作流，不负责训练。
- 一个 OpenAI 兼容模型端点：本地 vLLM、Ollama 或供应商 API 均可。

目录约定

```
ai-stack-lab/
├── distill/          # 蒸馏数据、训练、评测
├── workflow/         # LangGraph / n8n / Dify
├── mcp_server/       # FastMCP 服务
├── tests/            # 固定评测集
├── .env.example      # 只有变量名
└── README.md         # 架构、运行、结果
```

一次建好三个环境

```
sudo apt update
sudo apt install -y git build-essential
curl -LsSf https://astral.sh/uv/install.sh | sh

mkdir ai-stack-lab && cd ai-stack-lab
uv init --python 3.11
uv add torch transformers datasets accelerate \
  bitsandbytes sentencepiece
uv add langgraph langchain-openai \
  langchain-mcp-adapters
uv add "mcp[cli]" fastapi uvicorn
uv add --dev pytest ruff

# 先确认 CUDA，而不是直接开训
uv run python - <<'PY'
import torch
print(torch.__version__)
print(torch.cuda.is_available())
print(torch.cuda.get_device_name(0))
PY
```

> **版本策略：**第一次跑通后执行 `uv lock` 并提交 `uv.lock`。教程代码是可改造参考；SDK 版本更新时，以官方变更说明为准，不要长期裸用 `latest`。

贯穿三部分的验收基线

层	至少记录	失败判定
模型	任务正确率、延迟、显存、样本	仅看 loss，不做任务集
工作流	每节点输入/输出、重试、耗时	失败后无状态与原因
MCP	工具 schema、权限、调用日志	任意路径/动作均可访问

蒸馏不是“压缩按钮”，而是把教师分布迁移给学生

硬标签只告诉学生“哪个 token 是对的”；教师 logits 经过温度缩放后形成 soft targets，会透露“其他 token 相对有多合理”。这类暗知识正是蒸馏的价值。

$$p_T(i) = \mathrm{softmax}(z_{teacher}/T), \quad q_T(i) = \mathrm{softmax}(z_{student}/T)$$

$$L = \alpha \cdot CE(y, q) + (1 - \alpha) \cdot T^2 \cdot KL(p_T \,||\, q_T)$$

Temperature 温度

T = 1 接近原分布；**T > 1** 会“摊平”概率，让次优 token 的关系更可见。T 太高则信号过平。先从 2 开始。

KL 散度

衡量教师分布与学生分布的差异。训练时最小化前向 KL，使学生覆盖教师认为合理的 token。相同分布时为 0。

为什么乘 T^2

温度改变 softmax 梯度尺度；乘 T^2 用来补偿梯度变小，让不同温度下蒸馏项权重更可比较。

Teacher-Student 范式

教师冻结、只前向；学生接收 CE 与 KL 梯度。白盒要求能读取教师 logits，且 token 对齐。

因果语言模型要做的三处对齐

1. **同一词表：**最省事的是选择同系列、同 tokenizer 的教师与学生。不同词表不能直接逐 token 做 KL。
2. **预测位置右移：**位置 t 的 logits 预测 token t+1，因此比较 `logits[:, :-1]`，掩码来自 `labels[:, 1:]`。
3. **只算有效 token：**padding 或被设为 `-100` 的位置不能进入 KL 平均值。

> **白盒与黑盒的边界：**能拿 logits / 隐层就做白盒；只能调用聊天 API，就让教师生成“答案数据”再对学生做监督微调（SFT）。黑盒蒸馏损失了完整概率分布，但工程上常更容易。

12GB 显存下先做 1.7B → 0.6B 的同词表蒸馏

目标	优先方法	原因
让模型学领域语料	继续预训练（CPT）	改变知识与语言分布
让模型遵循任务格式	SFT / LoRA	成本低，目标直接
让输出更符合偏好	DPO 等偏好对齐	使用成对偏好信号
把大模型能力迁到小模型	蒸馏	优化成本、延迟、吞吐
模型已够小，只需专用化	微调	没有必要再降容量

保守起点

项	建议值	备注
Teacher	Qwen3-1.7B	FP16、冻结、eval
Student	Qwen3-0.6B	全参训练起步
序列长度	512	OOM 时先降到 384/256
micro batch	1	累积 16 步
α / T	0.5 / 2.0	需在验证集调参
优化器	paged_adamw_8bit	降低状态显存
学习率	2e-5	暖启动 3%
其他	FP16 + gradient checkpoint	4070 不要假定 BF16

显存降级顺序

1. 序列 512 → 384 → 256。
2. KL 分块 64 → 32。
3. 开启 checkpoint / 8-bit optimizer。
4. 离线缓存教师 top-k logits。
5. 学生改 LoRA；最后才换更小模型。

何时停止蒸馏

- 学生仍达不到任务最低线。
- 吞吐收益不足以覆盖训练成本。
- 教师错误被学生放大。
- 许可证或 API 条款不允许派生训练。

容量现实：教师与学生同时驻留 GPU，加上大词表 logits 会形成峰值显存。以下实现把 KL 按序列块计算；仍 OOM 时优先缩短序列，不要先降低数值精度到不稳定状态。

先用小而干净的数据验证损失，再扩大训练

准备 `distill/train.jsonl`，每行只有一个 `text` 字段。第一轮用 500-2,000 条领域文本即可验证 loss、显存和保存恢复；正式实验再扩到数万条。

```
{"text":"用户：订单 A1001 到哪里了？\n助手：我会先查询订单状态。"}
{"text":"用户：如何申请退款？\n助手：请提供订单号；是否可退以退款政策为准。"}
```

distill/train_distill.py · 模型与数据

```
import torch
import torch.nn.functional as F
from datasets import load_dataset
from transformers import (
    AutoModelForCausalLM, AutoTokenizer,
    DataCollatorForLanguageModeling, Trainer, TrainingArguments,
)

TEACHER = "Qwen/Qwen3-1.7B"
STUDENT = "Qwen/Qwen3-0.6B"
MAX_LEN = 512

# 直接比较词表分布，所以必须同 tokenizer / vocab。
tok = AutoTokenizer.from_pretrained(STUDENT, use_fast=True)
if tok.pad_token is None:
    tok.pad_token = tok.eos_token

def encode(batch):
    return tok(
        batch["text"], truncation=True,
        max_length=MAX_LEN, add_special_tokens=True,
    )

raw = load_dataset("json", data_files="distill/train.jsonl")["train"]
ds = raw.map(encode, batched=True, remove_columns=raw.column_names)
ds = ds.train_test_split(test_size=0.05, seed=42)
collator = DataCollatorForLanguageModeling(tok, mlm=False)

teacher = AutoModelForCausalLM.from_pretrained(
    TEACHER, torch_dtype=torch.float16,
    low_cpu_mem_usage=True,
)
teacher.requires_grad_(False)
teacher.eval()

student = AutoModelForCausalLM.from_pretrained(
    STUDENT, torch_dtype=torch.float16,
    low_cpu_mem_usage=True,
)
student.config.use_cache = False
```

> **先做词表断言：** `teacher.config.vocab_size == student.config.vocab_size == len(tok)`。如果不相等，不能直接做逐词表 KL；改用黑盒生成、隐层映射或词表映射方案。

自定义 Trainer 把 CE 与分块 KL 合在一起

```
class DistillTrainer(Trainer):
    def __init__(self, *args, teacher, alpha=0.5,
                 temperature=2.0, kd_chunk=64, **kwargs):
        super().__init__(*args, **kwargs)
        self.teacher = teacher.to(self.args.device)
        self.alpha = alpha
        self.temperature = temperature
        self.kd_chunk = kd_chunk

    def compute_loss(self, model, inputs, return_outputs=False,
                     num_items_in_batch=None):
        # Student 自带 causal LM CE; labels 中 pad 已是 -100。
        student_out = model(**inputs)
        ce = student_out.loss

        teacher_inputs = {
            k: v for k, v in inputs.items() if k != "labels"
        }
        with torch.no_grad():
            teacher_out = self.teacher(**teacher_inputs)

        # 位置 t 的 logits 预测 labels[t+1]。
        s_logits = student_out.logits[:, :-1, :]
        t_logits = teacher_out.logits[:, :-1, :]
        valid = inputs["labels"][:, 1:].ne(-100)
        T = self.temperature

        kd_sum = torch.zeros((), device=s_logits.device,
                             dtype=torch.float32)
        token_count = valid.sum().clamp_min(1)

        # 避免一次生成 [B, L, V] 的多个 FP32 softmax 张量。
        for start in range(0, s_logits.size(1), self.kd_chunk):
            end = start + self.kd_chunk
            mask = valid[:, start:end]
            if not mask.any():
                continue
            s_logp = F.log_softmax(
                s_logits[:, start:end].float() / T, dim=-1
            )
            t_prob = F.softmax(
                t_logits[:, start:end].float() / T, dim=-1
            )
            per_token = F.kl_div(
                s_logp, t_prob, reduction="none"
            ).sum(dim=-1)
            kd_sum = kd_sum + (per_token * mask).sum()

        kd = kd_sum / token_count
        loss = self.alpha * ce + \
               (1.0 - self.alpha) * (T ** 2) * kd

        # 日志可另外用 callback 记录 ce / kd。
        return (loss, student_out) if return_outputs else loss
```

为什么教师不用 labels
只需要 logits；不计算教师 CE，减少无用工作。

为什么转 float
softmax / KL 用 FP32 更稳；分块限制临时显存。

用短程冒烟测试确认显存，再开始正式训练

续写 distill/train_distill.py

```
args = TrainingArguments(
    output_dir="outputs/qwen3-0.6b-distilled",
    per_device_train_batch_size=1,
    per_device_eval_batch_size=1,
    gradient_accumulation_steps=16,
    learning_rate=2e-5,
    num_train_epochs=1,
    warmup_ratio=0.03,
    lr_scheduler_type="cosine",
    fp16=True,
    gradient_checkpointing=True,
    optim="paged_adamw_8bit",
    logging_steps=5,
    eval_strategy="steps",
    eval_steps=100,
    save_steps=100,
    save_total_limit=2,
    report_to="none",
    remove_unused_columns=False,
)

trainer = DistillTrainer(
    model=student, teacher=teacher,
    args=args,
    train_dataset=ds["train"],
    eval_dataset=ds["test"],
    data_collator=collator,
    processing_class=tok,
    alpha=0.5, temperature=2.0, kd_chunk=64,
)
trainer.train()                # 续训: resume_from_checkpoint=True
trainer.save_model(args.output_dir)
tok.save_pretrained(args.output_dir)
```

两阶段运行

```
# 1) 冒烟: 临时在 TrainingArguments 加 max_steps=20
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True \
  uv run python distill/train_distill.py

# 2) 观察显存; 无 OOM / NaN 后移除 max_steps
watch -n 1 nvidia-smi
```

评测不是“蒸馏 loss 更低”

指标	方法	必须对比
任务质量	固定 100-500 条问答 / 分类集	教师、原学生、蒸馏学生
延迟	相同硬件、prompt、输出长度	首 token、tokens/s
资源	峰值显存、模型大小	加载与运行峰值
鲁棒性	越权、空输入、超长输入	失败类型和恢复

常见问题：loss NaN 先把学习率降到 1e-5、检查 mask 与 FP32 KL；OOM 先降序列 / chunk；学生变得更像教师但任务分数下降，通常是教师错误、数据偏差或 α/T 不合适。

拿不到 logits 时，把蒸馏改造成合成数据工程

种子问题 → 教师生成 → 规则 / 模型过滤 → SFT 学生

generate_teacher_data.py

```
import json, os, time
from openai import OpenAI

client = OpenAI(
    base_url=os.environ["TEACHER_BASE_URL"],
    api_key=os.environ["TEACHER_API_KEY"],
)
model = os.environ["TEACHER_MODEL"]

with open("distill/prompts.jsonl", encoding="utf-8") as src, \
     open("distill/sft.jsonl", "a", encoding="utf-8") as out:
    for line in src:
        item = json.loads(line)
        prompt = item["prompt"].strip()
        if not prompt:
            continue
        resp = client.chat.completions.create(
            model=model,
            temperature=0.2,
            max_tokens=512,
            messages=[
              {"role":"system", "content":
               "你是客服助手。未知信息必须明确说明，不得编造订单状态。"},
              {"role":"user", "content": prompt},
            ],
        )
        answer = resp.choices[0].message.content.strip()
        if 20 <= len(answer) <= 2000:   # 只是第一层过滤
            out.write(json.dumps({"messages":[
              {"role":"user", "content":prompt},
              {"role":"assistant", "content":answer},
            ]}, ensure_ascii=False) + "\n")
        time.sleep(0.2)
```

质量过滤

- 去重 prompt 与答案。
- 规则查 PII、密钥、URL、空洞套话。
- 检验 JSON / 引用 / 业务约束。
- 抽样人工审阅至少 100 条。
- 保留教师模型、prompt 版本与时间。

合规门槛

- 确认教师模型许可证允许该用途。
- 确认 API 条款允许用输出训练模型。
- 未经授权不要上传客户私有数据。
- 可删除、可追溯、可复现。
- 对高风险领域保留人工审核。

不要把“教师说过”当真值：合成答案仍会幻觉。最佳做法是让问题来自真实任务分布，让答案能被规则、数据库或人工验证，并留一份从未参与训练的测试集。

工作流是可观察的状态机，不是无限循环的聊天机器人

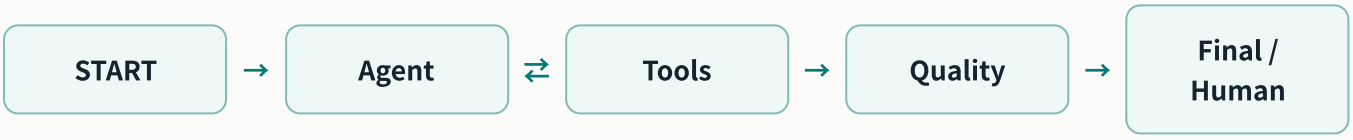
State 状态
消息、订单号、重试次数、审核结果。是节点间唯一可信的“共享记忆”。

Node 节点
一个清晰动作：分类、调用模型、查库、质检。输入状态，返回局部更新。

Edge 边
固定边表示确定顺序；条件边根据状态路由到工具、重试、人工或结束。

> 把 LLM 放在需要语义判断的地方；把权限、金额、重试、写入和结束条件留给确定性代码。

客服 Agent 的推荐拓扑



风险	控制方式	验收点
模型循环调用工具	recursion_limit / attempt	到上限可解释退出
重复写数据库	幂等键 / upsert	同一请求不重复产生副作用
工具参数越权	schema + allowlist + scope	非法参数被拒绝并记录
退款 / 发信等高风险动作	interrupt / 人工审批	未经批准绝不执行
节点失败	分级重试与 fallback	状态、原因、重试次数可见

为什么选 LangGraph：Graph API 用显式 State、Node、Edge 组织执行；编译阶段检查图结构，还可接 checkpointer 与断点。它比“一个大 prompt + while 循环”更适合作品集和客户交付。

完整示例：带工具调用与条件分支的客服 Agent

workflow/support_graph.py • 状态、工具、模型

```
import json, os
from typing import Literal
from langchain_core.messages import HumanMessage, SystemMessage
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI
from langgraph.graph import MessagesState, StateGraph, START, END
from langgraph.prebuilt import ToolNode

# 演示数据；生产中改为只读数据库查询。
ORDERS = {
    "A1001": {"status": "已发货", "carrier": "SF123456"},
    "A1002": {"status": "待付款", "carrier": None},
}

@tool
def lookup_order(order_id: str) -> str:
    """按订单号查询订单状态。只读，不修改订单。"""
    row = ORDERS.get(order_id.upper())
    if row is None:
        return json.dumps({"found": False}, ensure_ascii=False)
    return json.dumps({"found": True, "order_id": order_id,
                       **row}, ensure_ascii=False)

tools = [lookup_order]
llm = ChatOpenAI(
    model=os.getenv("CHAT_MODEL", "local-model"),
    base_url=os.getenv("OPENAI_BASE_URL", "http://127.0.0.1:8000/v1"),
    api_key=os.getenv("OPENAI_API_KEY", "local-not-secret"),
    temperature=0,
)
model_with_tools = llm.bind_tools(tools)

class SupportState(MessagesState):
    attempts: int
    quality: str
    status: str
    final: str

SYSTEM = """你是订单客服。涉及订单状态时必须先调用查询工具。
只使用工具返回的数据，不得编造。退款、取消、改地址只给说明，不执行。"""

def agent_node(state: SupportState):
    reply = model_with_tools.invoke(
        [SystemMessage(content=SYSTEM)] + state["messages"]
    )
    return {"messages": [reply],
            "attempts": state.get("attempts", 0) + 1}

def route_after_agent(state: SupportState) -> Literal["tools", "quality"]:
    return "tools" if state["messages"][-1].tool_calls else "quality"
```

> **ToolNode 的职责：**读取最后一条 AIMessage 里的 tool_calls，执行匹配工具，再把 ToolMessage 写回 messages。随后回到 agent，让模型基于真实结果生成答案。

质检节点把失败、重写与人工审核显式化

续写 workflow/support_graph.py

```
def quality_node(state: SupportState):
    answer = str(state["messages"][-1].content)
    user_text = " ".join(
        str(m.content) for m in state["messages"]
        if isinstance(m, HumanMessage)
    )
    risky = any(x in user_text for x in ["退款", "取消", "改地址"])
    adequate = len(answer) >= 40 and not any(
        x in answer for x in ["我猜", "可能已经", "大概是"]
    )
    if risky:
        return {"quality": "review"}
    if adequate:
        return {"quality": "pass"}
    # 失败时给模型一个明确修订要求，而不是静默重跑。
    return {
        "quality": "retry",
        "messages": [HumanMessage(
            content="请基于已有工具结果重写；不得猜测，答案至少40字。"
        )],
    }

def route_quality(state: SupportState) -> Literal["agent", "final", "human"]:
    if state["quality"] == "review":
        return "human"
    if state["quality"] == "pass":
        return "final"
    return "agent" if state["attempts"] < 3 else "human"

def final_node(state: SupportState):
    return {"status": "done",
            "final": str(state["messages"][-1].content)}

def human_node(state: SupportState):
    return {"status": "pending_human_review",
            "final": "需要人工审核，工作流未执行任何变更。"}

builder = StateGraph(SupportState)
builder.add_node("agent", agent_node)
builder.add_node("tools", ToolNode(tools, handle_tool_errors=True))
builder.add_node("quality", quality_node)
builder.add_node("final", final_node)
builder.add_node("human", human_node)

builder.add_edge(START, "agent")
builder.add_conditional_edges("agent", route_after_agent)
builder.add_edge("tools", "agent")
builder.add_conditional_edges("quality", route_quality)
builder.add_edge("final", END)
builder.add_edge("human", END)
graph = builder.compile()

if __name__ == "__main__":
    result = graph.invoke(
        {"messages": [HumanMessage(
            content="查询订单 A1001，并告诉我现在的状态。"
        )], "attempts": 0},
        config={"recursion_limit": 12},
    )
    print(result["status"], result["final"])
```

> **生产改造：**把人工节点换成 `interrupt()`，并用持久化 checkpointer 保存线程。任何写操作必须放在审批之后，且用幂等键保护；不要让模型自己决定是否“跳过审批”。

先测每个路由，再测整条链；不要只录一次成功演示

最小测试矩阵

输入	预期路径	断言
“查询 A1001”	agent → tools → agent → quality → final	含“已发货”，不含猜测
“查询 X9999”	同上	明确未找到，不编状态
“退款 A1001”	… → quality → human	status=pending_human_review
工具抛异常	ToolMessage 错误 → agent	可解释失败，不崩图
模型连续输出太短	quality → agent（最多 3 次）	最终转人工

pytest 骨架：用假模型隔离 API 波动

```
# tests/test_routes.py
from langchain_core.messages import AIMessage, HumanMessage
from workflow.support_graph import route_after_agent, route_quality

def test_tool_route():
    state = {"messages": [AIMessage(content="", tool_calls=[{
      "name":"lookup_order", "args":{"order_id":"A1001"}, "id":"1"
    }])], "attempts": 1}
    assert route_after_agent(state) == "tools"

def test_review_route():
    state = {"messages": [HumanMessage(content="退款 A1001")],
             "attempts": 1, "quality": "review"}
    assert route_quality(state) == "human"

def test_retry_cap():
    state = {"messages": [AIMessage(content="短")],
             "attempts": 3, "quality": "retry"}
    assert route_quality(state) == "human"
```

观测字段

- request_id / thread_id
- node、开始/结束时间
- 模型与 prompt 版本
- tool 名与脱敏参数
- token、费用、错误类型

上线门禁

- P95 延迟与成本上限
- 失败重试不产生副作用
- 敏感动作均需确认
- 工具 allowlist
- 日志不写原始密钥 / PII

作品集重点：README 中画出状态图，附 5 条失败用例和一张“原始模型 vs 工作流”的可靠性对比。客户通常更在乎失败怎么收口，而不是节点动画多漂亮。

n8n 优化“系统集成”，Dify 优化“AI 应用编排”

维度	n8n	Dify	LangGraph
最擅长	触发器、SaaS、ETL	RAG、Prompt、模型节点	复杂状态、代码控制
上手速度	快	快	较慢
可定制性	中	中	高
适合客户	运营 / 业务自动化	知识库 / 聊天应用	核心产品 / 复杂 Agent
常见风险	流程膨胀、凭证治理	平台耦合、复杂逻辑受限	开发与运维成本

n8n：自动化报表

Schedule → DB/API → Code → LLM

- 定时拉取销售 / 工单数据。
- 代码节点清洗并计算指标。
- LLM 只写解释，不负责算数。
- 人工确认后发邮件 / 飞书。

Dify：客服 RAG

Question → Retrieve → LLM → Answer

- 文档分块、召回与引用。
- IF/ELSE 判断低置信度。
- 敏感问题转人工。
- 通过 API 嵌入客户系统。

搭建顺序

1. **先画 I/O：**触发、数据来源、最终交付物、失败通知。
2. **再做 happy path：**只连 1 个数据源与 1 个输出。
3. **补失败分支：**超时、空数据、解析失败、限流、人工接管。
4. **最后接真实凭证：**最小权限，使用平台凭证库，不写在节点文本。

选型原则：“80% 是 SaaS 接口”选 n8n；“80% 是知识库与模型节点”选 Dify；有长状态、复杂重试、精细测试或核心 IP，选 LangGraph。也可用 n8n 做外层触发，LangGraph 做核心决策。

按“可验收工作包”报价，不按节点数或代码行报价

项目形态	推荐技术	P0 验收
客服 RAG 工作流	Dify + Qdrant + API；复杂审批加 LangGraph	30 个固定问题；有引用；低置信转人工
自动化经营报表	n8n + 数据库/API + Python + LLM	按时出报表；数字可复算；失败告警
内部工具 Agent	LangGraph + MCP + FastAPI	3 个只读工具；审计日志；写操作需确认

报价拆成五块

发现与方案
数据源、权限、验收集、架构。

P0 实现
一条主路径、最少集成、演示。

生产加固
鉴权、重试、审计、备份。

部署交付
Docker、文档、培训、移交。

持续运维
监控、额度、模型与接口变化。

第三方成本
模型 API、短信、云资源另列。

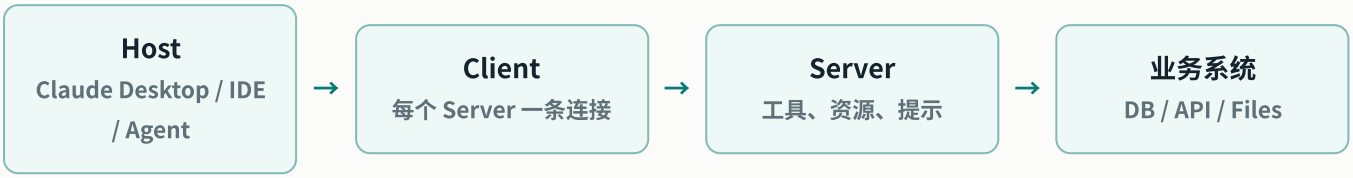
起步试报价框架

工作包	练习区间（人民币）	边界
付费需求梳理 + 原型	¥500-1,500	1 次访谈、流程图、假数据 demo
P0 小项目	¥2,000-6,000	1-2 数据源、1 输出、固定验收集
生产化扩展	¥6,000 起，按风险重估	鉴权、权限、审计、并发、运维
月度维护	¥500-2,000+	明确小时数与响应窗口

以上是用于形成报价习惯的建议区间，不是市场均价。首单可以收窄范围，不要无限降价；价格由数据源数量、权限风险、并发、SLA、是否驻场和第三方接口稳定性调整。

防止范围失控：合同 / 报价单写清输入样例、验收题集、最大迭代次数、部署环境、第三方费用、数据责任和“不包含项”。新增数据源、新权限、新渠道都走变更单。

MCP 标准化“模型如何发现并调用能力”，不替代业务工作流



Host

管理用户体验、模型、权限、多个 Client 和会话。它决定哪些服务可见、何时向用户确认。

Client

嵌在 Host 内，负责协议握手、能力发现、请求 / 响应与通知。一条 Client 通常对应一个 Server。

Server

把后端能力以 MCP primitives 暴露。Server 不应该默认信任模型或 Host。

三类核心能力

能力	用途	订单示例	副作用
Tools	可调用函数	lookup_order(order_id)	可有；需声明与防护
Resources	可读取上下文	policy://refund	原则上只读
Prompts	复用交互模板	support_reply(tone)	无直接副作用

与 REST 的关系：MCP 不要求你重写业务层。最稳的结构是“领域服务”提供查询 / 写入函数，REST 与 MCP 各做一层适配。这样权限、验证、测试可以共享，协议变化也不会侵入核心逻辑。

安全边界

- 把来自资源、网页、文件和工具结果的内容都当作不可信数据；不要执行其中的指令。
- 工具最小化：一个 tool 做一个明确动作；参数 schema 越窄越好。
- 读写分离：查询工具与修改工具分开；修改工具要求确认、幂等键与审计。
- 远程 MCP 需要 HTTPS、鉴权、速率限制与租户隔离；不要直接暴露开发服务。

本地优先 stdio；新远程服务优先 Streamable HTTP

传输	适合	优点	注意
stdio	本地桌面、IDE、单用户	Client 启动子进程；无需监听端口	stdout 只能传协议；日志写 stderr
Streamable HTTP	远程、多客户端、VPS	单 HTTP 端点；可返回 JSON 或事件流	HTTPS、鉴权、Origin 校验、限流
HTTP + SSE（旧）	兼容旧客户端	已有生态仍可能支持	已是遗留路径；新项目不要作为默认

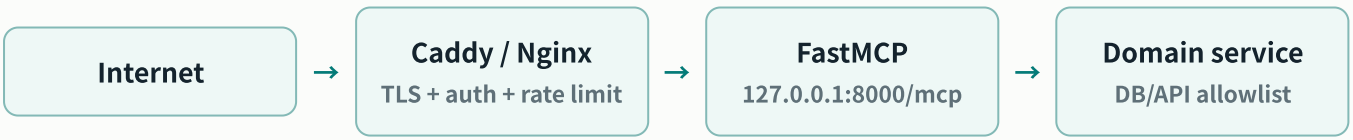
stdio 生命周期

1. Host 按配置启动 Server。
2. JSON-RPC 经 stdin/stdout 交换。
3. Server 仅向 stderr 写日志。
4. Host 关闭 stdin，Server 应退出。

Streamable HTTP 生命周期

1. Client 向单一 MCP 端点 POST。
2. 响应可为 JSON 或 SSE 流。
3. 按协议协商 session / version。
4. 网关负责 TLS、认证与配额。

VPS 部署建议



> **不要把 SSE 这个词混为一谈：**旧“HTTP + SSE”是双端点式遗留传输；Streamable HTTP 的响应内部仍可使用 Server-Sent Events 做流式传输。新远程集成目标应是 Streamable HTTP。

选择口诀

- **同一台机器、由客户端启动：**stdio。
- **跨机器、多个调用方、需要网关：**Streamable HTTP。
- **第三方客户端只支持旧协议：**临时保留兼容层，并设置迁移计划。

从零写一个有 Tool、Resource、Prompt 的 MCP Server

安装与目录

```
mkdir -p mcp_server
uv add "mcp[cli]"
# 下面保存为 mcp_server/server.py
```

server.py • 数据与 Tool

```
import argparse
import json
import sqlite3
from pathlib import Path
from mcp.server.fastmcp import FastMCP

DB = Path(__file__).with_name("orders.db")
mcp = FastMCP("OrderOps", json_response=True)

def connect():
    conn = sqlite3.connect(DB)
    conn.row_factory = sqlite3.Row
    return conn

def init_db():
    with connect() as conn:
        conn.execute("""CREATE TABLE IF NOT EXISTS orders (
          order_id TEXT PRIMARY KEY,
          status TEXT NOT NULL,
          carrier TEXT
        )""")
        conn.execute(
          "INSERT OR IGNORE INTO orders VALUES (?, ?, ?)",
          ("A1001", "已发货", "SF123456"),
        )

@mcp.tool()
def lookup_order(order_id: str) -> dict:
    """按订单号查询状态。该工具只读，不修改订单。"""
    clean = order_id.strip().upper()
    if not clean or len(clean) > 32:
        raise ValueError("order_id 格式不正确")
    with connect() as conn:
        row = conn.execute(
          "SELECT order_id, status, carrier FROM orders WHERE order_id = ?",
          (clean,),
        ).fetchone()
    if row is None:
        return {"found": False, "order_id": clean}
    return {"found": True, **dict(row)}
```

> **为什么用参数化 SQL：**工具参数来自模型，必须当成不可信输入。类型注解生成 schema 只是第一层；业务格式校验、参数化查询、权限和结果脱敏仍需自己实现。

Resource 提供只读上下文，Prompt 提供复用模板

续写 mcp_server/server.py

```
@mcp.resource("policy://refund")
def refund_policy() -> str:
    """读取演示用退款政策。"""
    return json.dumps({
      "version": "demo-1",
      "rules": [
        "未发货订单可提交退款申请",
        "已发货订单需先进入退货流程",
        "最终结果必须由业务系统确认",
      ],
    }, ensure_ascii=False)

@mcp.prompt()
def support_reply(order_id: str, tone: str = "简洁") -> str:
    """生成客服回复任务模板。"""
    return (
      f"请先调用 lookup_order 查询 {order_id}。"
      f"用{tone}语气回答；未知信息不得猜测；"
      "如果涉及退款，只解释 policy://refund，不执行操作。"
    )

def main():
    init_db()
    parser = argparse.ArgumentParser()
    parser.add_argument(
      "--transport",
      choices=["stdio", "streamable-http"],
      default="stdio",
    )
    args = parser.parse_args()
    mcp.run(transport=args.transport)

if __name__ == "__main__":
    main()
```

本地调试

```
# 方式 A：官方 MCP Inspector
npx -y @modelcontextprotocol/inspector \
  uv --directory "$PWD" run python mcp_server/server.py

# 方式 B：启动 Streamable HTTP
uv run python mcp_server/server.py --transport streamable-http
# Inspector 中连接 http://127.0.0.1:8000/mcp
```

Inspector 必测

- [ ] 列出 tools / resources / prompts，名称和 schema 正确。
- [ ] `lookup_order("A1001")` 返回 found=true；未知订单返回 false。
- [ ] 空值、超长值、特殊字符被拒绝；服务不崩溃。
- [ ] 读取 `policy://refund`；获取 `support_reply` 模板。
- [ ] stdio 模式下 stdout 没有调试 print；日志只写 stderr。

> **版本提醒：**官方 Python SDK 在 2026 年经历 v1 → v2 迁移。先固定已验证版本；若 `mcp.run` 参数或 CLI 变化，按对应版本 README 迁移，不要同时改 SDK、协议和业务代码。

区分“已验证的配置格式”和“产品并未公开的能力”

Claude Desktop：stdio 配置

macOS 配置文件通常位于 `~/Library/Application Support/Claude/claude_desktop_config.json`；Windows 位于 `%APPDATA%\Claude\claude_desktop_config.json`。使用绝对路径，保存后完全退出并重启应用。

```
{
  "mcpServers": {
    "order-ops": {
      "command": "/ABSOLUTE/PATH/TO/uv",
      "args": [
        "--directory",
        "/ABSOLUTE/PATH/TO/ai-stack-lab",
        "run",
        "python",
        "mcp_server/server.py"
      ]
    }
  }
}
```

排错

- `which uv` 取得绝对路径。
- 先在终端执行同一 command + args。
- JSON 不能有注释或尾逗号。
- Server 日志写 stderr。
- 应用环境不一定继承 shell PATH。

环境变量

非敏感配置可放 `env`；真实密钥使用系统密钥链、客户端安全凭证能力或服务端凭证。不要把密钥提交到仓库，也不要在教程示例中粘贴真实值。

Muse：当前不要编造配置文件

> **当前结论：**现有 Muse 文档说明连接器只提供其 skill 文档列出的命令；对于任意应用的 MCP Server，没有文档化的 `mcpServers` 配置文件、Server URL 输入框或传输设置。因此本教程不能给出一个“可用的 Muse MCP 配置文件”。

你仍可以用下面两种方式完成同一学习目标：

1. **协议验证：**用 MCP Inspector 或 Claude Desktop 接入自建 Server。
2. **产品集成：**把 MCP Client 放进你自己的 LangGraph / FastAPI 应用；Muse 仅在未来明确公开该能力后，再按官方文档接入。

> **不要混淆：**Muse 的内置 connector / skill 与用户自建 MCP Server 不是同一能力。不存在文档的入口，不能靠猜文件名或设置路径“接入”。

把 MCP 作为 LangGraph 的“工具供应层”

MCP 负责发现与调用工具；LangGraph 负责状态、条件分支、重试与人工审核。两者是互补关系。

workflow/mcp_graph.py

```
import asyncio, os, sys
from langchain_openai import ChatOpenAI
from langchain_mcp_adapters.client import MultiServerMCPClient
from langgraph.graph import MessagesState, StateGraph, START
from langgraph.prebuilt import ToolNode, tools_condition

async def main():
    client = MultiServerMCPClient({
      "orders": {
        "command": sys.executable,
        "args": ["mcp_server/server.py"],
        "transport": "stdio",
      }
    })
    mcp_tools = await client.get_tools()

    model = ChatOpenAI(
      model=os.getenv("CHAT_MODEL", "local-model"),
      base_url=os.getenv("OPENAI_BASE_URL",
                         "http://127.0.0.1:8000/v1"),
      api_key=os.getenv("OPENAI_API_KEY", "local-not-secret"),
      temperature=0,
    ).bind_tools(mcp_tools)

    async def call_model(state: MessagesState):
        reply = await model.ainvoke(state["messages"])
        return {"messages": [reply]}

    builder = StateGraph(MessagesState)
    builder.add_node("agent", call_model)
    builder.add_node("tools", ToolNode(
      mcp_tools, handle_tool_errors=True
    ))
    builder.add_edge(START, "agent")
    builder.add_conditional_edges("agent", tools_condition)
    builder.add_edge("tools", "agent")
    graph = builder.compile()

    result = await graph.ainvoke({
      "messages": [("system",
        "订单状态必须调用工具；只使用工具结果，不得猜测。"),
        ("user", "订单 A1001 现在是什么状态？")]
    }, config={"recursion_limit": 8})
    print(result["messages"][-1].content)

if __name__ == "__main__":
    asyncio.run(main())
```

端到端验收

工具来自 MCP 动态发现 | 不存在的订单不幻觉 | 工具异常可收口 | 8 步后强制结束 | 日志可按 request_id 追踪

> **下一步：**把 stdio 配置替换为远程 `{"url":"https://你的域名/mcp","transport":"http"}` 前，先完成 TLS、鉴权、权限隔离与速率限制。不要把本地无鉴权 demo 直接暴露到公网。

每周都产出可展示资产，第四周合成一个接单作品

周	三天安排	可展示产出
第 1 周 **蒸馏**	Day 1：soft targets / T / KL 手算与小张量测试。 Day 2：1.7B → 0.6B 冒烟、OOM 调整。 Day 3：固定评测集与速度 / 显存对比。	`train_distill.py` 实验表格 损失曲线 + 结论
第 2 周 **工作流**	Day 1：画状态图与验收矩阵。 Day 2：完成 LangGraph 工具循环、质检、人工分支。 Day 3：pytest、日志、失败演示。	状态图 `support_graph.py` 5 个测试用例
第 3 周 **MCP**	Day 1：Host / Client / Server 与三类能力。 Day 2：FastMCP Server + Inspector。 Day 3：Claude Desktop + LangGraph Client 接入。	可运行 MCP Server Inspector 截图 工具 schema 文档
第 4 周 **产品化**	Day 1：合并订单客服 demo。 Day 2：Docker / VPS、TLS、鉴权与监控。 Day 3：README、演示视频、报价单与复盘。	一键启动 demo 架构图 + README 报价模板

第四周的作品结构

```
order-agent-portfolio/
├── README.md                # 业务问题、架构、指标、限制
├── distill/                 # 训练脚本与 eval_results.json
├── workflow/                # LangGraph 图与路由测试
├── mcp_server/              # Tool / Resource / Prompt
├── tests/                   # 固定验收题与失败用例
├── docker-compose.yml       # 服务编排
├── .env.example             # 变量名，不含秘密
└── docs/
    ├── architecture.svg
    ├── acceptance.md
    └── pricing-scope.md
```

演示视频 3 分钟

1. 20 秒：客户问题。
2. 40 秒：架构与边界。
3. 60 秒：成功流程。
4. 40 秒：失败 / 人审。
5. 20 秒：指标与交付物。

README 必须回答

- 为什么不是纯聊天机器人？
- 如何防幻觉与越权？
- 如何复现与评测？
- 12GB 显存如何取舍？
- 客户怎么验收？

VPS 负责接口与编排；4070 负责训练和可选推理

无 GPU 的普通 VPS

- 运行 FastAPI、LangGraph、MCP Server、数据库。
- 调用外部 OpenAI 兼容模型端点。
- 小模型可用 llama.cpp 量化推理，但先压测。
- 优先 2 vCPU / 4GB RAM 起步，按实测调整。

有 RTX 4070 的机器

- 训练 0.6B / 1.7B 与 7B QLoRA。
- 部署蒸馏后的 0.6B / 1.7B 更从容。
- 7B 推理使用 4-bit 量化并缩短上下文。
- 不要让训练与生产推理抢同一张卡。

公网链路

Client → HTTPS Gateway → App / MCP → Model + DB

上线项	最低要求	验收证据
网络	仅开放 80/443；后端绑定 loopback	端口扫描 / 防火墙规则
传输	TLS；远程 MCP 用 Streamable HTTP	证书与握手成功
认证	每客户独立身份 / token scope	越权请求 401/403
秘密	环境注入或密钥服务；不进镜像	仓库扫描无秘密
可靠性	超时、限流、重试、幂等	故障演练记录
数据	最小化、脱敏、保留期	删除与导出流程
观测	健康检查、结构化日志、告警	一次告警演示
回滚	镜像标签、数据库备份、旧版配置	回滚演练

安全重点：模型输出、MCP Resource、网页内容都可能含提示注入。它们只能作为数据进入受限流程，不能获得改变系统指令、扩大权限、读取秘密或选择新目标的能力。

最终“可交付”定义

新机器按 README 可启动；固定验收集通过；失败可解释并转人工；第三方费用可计算；客户能替换模型或数据源；你能在 30 分钟内回滚到上一版本。

把这份教程真正跑通：最后一页行动清单

权威资料入口

- **知识蒸馏实现思路**
  Hugging Face Transformers • Knowledge Distillation
- **在线生成式蒸馏**
  TRL • GKD Trainer
- **图状态、节点与边**
  LangGraph • Graph API
- **MCP 协议规范**
  Model Context Protocol • Specification
- **官方 Python SDK**
  modelcontextprotocol/python-sdk
- **低代码工作流**
  n8n • Build your first workflow
- **AI 应用工作流**
  Dify • Workflow documentation

开工前 10 项终检

01 固定 teacher / student 版本与 tokenizer。

02 先跑 20 步 smoke test，再开长训练。

03 记录峰值显存、吞吐、验证损失。

04 黑盒数据保留来源、许可与生成参数。

05 先画状态图，再写 LangGraph 节点。

06 工具调用覆盖成功、失败、超时与越权。

07 MCP SDK 锁版本；升级时重跑契约测试。

08 远程 MCP 上线前补 TLS、鉴权和限流。

09 报价写范围、验收、变更与第三方费用。

10 作品集同时展示结果、失败路径和取舍。

版本提醒：SDK、框架与客户端配置会变化。动手前以对应官方文档和锁定版本为准；本教程给出的是可改造基线，不替代目标版本的迁移说明。

BUILD • TEST • SHIP

先交付一个可靠的小闭环，再增加模型、工具与自动化。

你的第一个目标不是“全学会”，而是完成一个可演示、可复现、可报价的订单客服工作流。